

IMDB 电影评论情感分析实验报告

学号：PB23000270 组别：1 姓名：李佩优

2026 年 4 月 27 日

1 引言

本实验针对 IMDB 电影评论数据集进行二分类情感分析（正面/负面）。根据课程要求，我们分别实现了基于多头自注意力（Multi-Head Attention）和循环神经网络（RNN）的分类器，并对多种超参数进行搜索，以及完成若干附加实验，包括手动实现 MHA、位置编码消融、RoPE、因果 mask 和注意力权重可视化。

2 实验方法与模型架构

2.1 数据预处理与词表构建

数据集为 CSV 格式，包含 text 和 label 两列，共 50,000 条电影评论，正负样本各 25,000 条。评论文本的平均长度为 231 词，中位数 173 词，95% 分位数为 590 词。我们使用 `sklearn.model_selection.train_test_split` 按照 80% 训练、10% 验证、10% 测试的比例划分数据集（训练集 36,000 条，验证集 4,000 条，测试集 10,000 条）。

采用自定义词表：统计训练集词频，保留前 15,000 个高频词，其余替换为 `<unk>`，引入 `<pad>`（索引 0）和 `<unk>`（索引 1）。所有序列统一填充至长度 200，填充方向为左侧填充，并通过 `attention_mask` 区分有效 token 与填充。

同时实现了两种词表构建方式（见 `src/tokenizer.py`）：

- (1) 自定义词表：如上述方法构建。
- (2) 预训练 tokenizer：通过 `transformers.AutoTokenizer` 加载 GPT-2 等分词器，统一映射为连续 ID，默认使用自定义词表。

2.2 模型设计

2.2.1 Attention 分类器

模型结构见 `src/models/attention_model.py`，核心组件：

- 词嵌入层: `nn.Embedding(vocab_size, embed_dim)`, `padding_idx=0`。
- 正弦位置编码: `src/utils.py` 中的 `PositionalEncoding`, 使用标准 Transformer 位置编码公式。
- Transformer 编码器: 调用 `nn.TransformerEncoder`, 包含 `num_layers` 层, 每层为 `nn.TransformerEncoderLayer` (多头自注意力 + 前馈网络), 前馈网络隐藏层维度为 $4 \times d_{model}$ 。
- 池化与分类: 对有效 token 进行平均池化, 后接线性层输出 1 个 logit, 使用 `BCEWithLogitsLoss`。

前向传播流程:

$x = \text{Embedding}(\text{input_ids}) + \text{PositionalEncoding} \rightarrow \text{TransformerEncoder}(x, \text{src_key_padding_mask})$

2.2.2 RNN 分类器

模型结构见 `src/models/rnn_model.py`:

- 嵌入层同 Attention 模型。
- RNN 层: 支持 LSTM 或 GRU, 隐藏维度 `hidden_dim`, 层数 `num_layers`, 单向。
- 平均池化 (考虑 mask) 后接线性分类器。

2.2.3 手动实现的多头注意力

为完成附加实验, 在 `manual_mha.py` 中实现了 `ManualMultiheadAttention`, 完全遵循《Attention Is All You Need》中的缩放点积注意力:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + \text{mask} \right) V$$

手动实现包括 Q/K/V 线性投影、多头拆分与合并、softmax 归一化、以及 padding mask 和注意力 mask 的支持。该模块被 `ManualAttentionClassifier` 调用, 并与标准 PyTorch MHA 进行性能对比。

2.2.4 位置编码变体

- 无位置编码: 直接省略位置编码层。
- RoPE (旋转位置编码): 在 `src/utils.py` 中实现, 对每对相邻维度进行旋转操作, 保持相对位置信息。
- 因果注意力 mask: 通过上三角矩阵 mask 屏蔽未来 token, 实现自回归注意力。

2.3 训练与评估

训练使用 Adam 优化器,学习率 5×10^{-4} ,梯度裁剪阈值 1.0,损失函数为 BCEWithLogitsLoss。每个 epoch 计算训练损失,并在验证集上评估准确率和 F1 分数 (sklearn.metrics),最终在测试集上报告结果。共训练 5 个 epoch。

3 实验配置

基本超参数默认值见表1。

表 1: 默认超参数配置

参数	值
嵌入维度 embed_dim	256
注意力头数 num_heads	8
Transformer 层数 num_layers	4
RNN 隐藏维度 hidden_dim	256
最大序列长度 max_seq_len	200
词表大小 max_vocab_size	15000
批大小 batch_size	64
学习率 lr	5e-4
训练轮次 num_epochs	5
Dropout	0.1
优化器	Adam

所有实验可通过命令行参数灵活配置,核心入口为 main.py,自动调用 experiment_runner.py 中的 run() 函数,根据 ExperimentConfig 选择训练、超参搜索、对比或可视化模式。

4 实验结果

4.1 基础模型性能

表2列出了 Attention 与 RNN 在测试集上的结果。RNN (LSTM) 在准确率和 F1 分数上均略优于标准 Attention 模型,且训练损失下降更快,表明在该任务上 LSTM 对序列信息的捕捉能力较强。

表 2: 基础模型测试结果

模型	Accuracy	F1-score
Attention (Sinusoidal PE)	0.8385	0.8365
RNN (LSTM)	0.8589	0.8622

4.2 手动 MHA 与内置 MHA 对比

表3显示,手动实现的 MHA 在最佳验证 F1 上达到 0.8443,略微超过内置 `nn.MultiheadAttention` 的 0.8404, 差距在随机波动范围内, 证明手工实现正确且性能与 PyTorch 官方实现相当。

表 3: 手动 MHA vs. 内置 MHA (最佳验证 F1)

模型	最佳验证 F1
Sinusoidal PE (内置 MHA)	0.8404
Manual MHA (手工实现)	0.8443

4.3 位置编码影响分析

表4对比了有无正弦位置编码、RoPE 以及因果 mask 的性能。令人关注的是, **无位置编码**的模型取得了 0.8571 的最佳验证 F1, 显著高于带有正弦位置编码的 0.7865。这可能是因为情感分类对词序信息的需求有限, 且固定的正弦位置编码在小规模数据上可能干扰语义表达。RoPE 以 0.8301 的 F1 次之, 优于正弦位置编码但不及无位置编码的版本, 表明 RoPE 引入的相对位置信息对分类略有帮助, 但仍不如此时直接省略位置编码的方案高效。

表 4: 位置编码变体对比 (最佳验证 F1)

位置编码方式	最佳验证 F1
Sinusoidal PE	0.7865
无位置编码	0.8571
RoPE	0.8301

4.4 因果 mask 的影响

在分类任务上强制使用因果 mask (只允许关注当前位置之前的信息) 导致模型性能剧烈下降, 最佳验证 F1 仅有 0.4871, 而相同设置下的双向注意力 F1 为 0.7970 (见

表5)。分类任务需要综合利用整个序列的上下文信息，因果 mask 阻断了后文对前文的影响，严重损害了模型理解能力，这与预期一致。

表 5: 因果 mask vs. 双向注意力（最佳验证 F1）

注意力模式	最佳验证 F1
双向注意力（Sinusoidal PE）	0.7970
因果 mask	0.4871

4.5 超参数实验结果

针对注意力头数（num_heads）、嵌入维度（embed_dim）、模型层数（num_layers）、学习率（lr）和 Dropout 率（dropout）进行了网格搜索，在验证集上记录最佳 F1。

注意力头数 结果见表6。头数为 4 和 16 时性能较优（F1 分别为 0.8327 和 0.8425），而头数为 2 时表达能力有限，F1 降至 0.7859。头数为 8 时出现异常大幅下降（F1 仅 0.7109），可能由于该次运行的随机种子导致训练陷入尖锐极小值，或学习率与模型容量不匹配，需进一步检查训练曲线。总体而言，适当增加头数（如 16 头，每头维度 16）可以提升模型容量，但并非头数越多越好，4 头与 16 头性能接近，说明注意力头数对最终效果影响有限，存在较优范围。

表 6: 不同注意力头数的最佳验证 F1

num_heads	最佳验证 F1
2	0.7859
4	0.8327
8	0.7109
16	0.8425

嵌入维度 结果见表7。嵌入维度 128 和 256 均取得了较高的 F1（0.8517 和 0.8488），而嵌入维度 512 时模型完全失效（F1 仅 0.6664，基本等同于随机猜测）。这很可能是因为嵌入维度增大导致参数量剧增（尤其前馈网络维度 $=4 \times d_{model}$ ），在小规模数据上极易过拟合或优化困难；同时，固定的学习率 5×10^{-4} 对于大模型可能不适用，造成训练发散。这说明对于当前数据规模，128 或 256 维嵌入已足够，盲目增大模型容量反而有害。

表 7: 不同嵌入维度的最佳验证 F1

embed_dim	最佳验证 F1
128	0.8517
256	0.8488
512	0.6664

模型层数 结果见表8。2 层 Transformer (F1=0.8524) 表现最佳, 4 层稍低 (0.8417), 而 6 层模型完全崩溃 (F1=0.6664)。浅层网络在数据量有限时展现了更好的泛化能力; 过深的网络 (6 层) 参数量过大, 训练困难, 且可能因梯度消失或过拟合导致性能退化。该结果与嵌入维度的结论一致: 对于小型分类任务, 轻量级架构往往更为合适。

表 8: 不同模型层数的最佳验证 F1

num_layers	最佳验证 F1
2	0.8524
4	0.8417
6	0.6664

学习率 结果见表9。学习率为 0.001 时模型完全无法收敛 (F1=0.6664), 表明初始步长过大, 导致参数更新剧烈, 可能跳过了最优区域。学习率 5×10^{-4} 和 1×10^{-4} 均工作良好, 分别达到 0.8434 和 0.8457 的 F1, 其中更小的学习率 (1×10^{-4}) 取得了微弱的优势, 且训练过程更稳定。该实验说明, 当前模型对学习率较为敏感, 需设置在合适范围内 (建议 10^{-4} 量级)。

表 9: 不同学习率的最佳验证 F1

lr	最佳验证 F1
0.001	0.6664
0.0005	0.8434
0.0001	0.8457

Dropout 率 结果见表10。不使用 Dropout (0.0) 和标准 Dropout (0.1) 均能取得较好性能, F1 分别为 0.8455 和 0.8486, 两者差异在误差范围内。当 Dropout 增大至 0.2 时, 模型性能大幅下滑 (F1=0.7073), 训练和验证曲线出现明显波动, 表明过强的正则化限制了模型的学习能力, 尤其是在仅有 5 个 epoch 的训练设置下, 模型尚未充分拟

合前就被过度约束。因此，适度的 Dropout (0.1) 或不使用 Dropout 均可，强烈正则化不利于模型收敛。

表 10: 不同 Dropout 率的最佳验证 F1

dropout	最佳验证 F1
0.0	0.8455
0.1	0.8486
0.2	0.7073

5 分析与讨论

Attention vs. RNN RNN (LSTM) 在 IMDB 分类任务上取得了略优的性能，其训练收敛速度也更快（第 5 个 epoch 训练损失仅 0.1069，而 Attention 为 0.2408）。这可能因为 LSTM 天然的序列建模能力足以捕捉情感线索，而 Transformer 在小规模数据上容易过拟合，且可能对超参数更为敏感。但两者差距并不悬殊（F1 分别为 0.8622 和 0.8365），说明 Attention 模型也具有较强竞争力。

位置编码的必要性 注意到，去掉位置编码后模型性能反而大幅提升。我们认为原因有三：一是情感分类更依赖关键词而非词序，“not good”这类短语虽然涉及顺序，但平均池化已经能够捕获部分词袋特征；二是固定正弦编码可能与嵌入维度及数据分布不匹配，引入了不必要的归纳偏置；三是训练轮次较少，位置编码可能加剧了优化难度。这一现象表明，对于简单分类任务，位置编码并非总是必须，需谨慎选用。

RoPE 的改进 RoPE 相较于正弦编码提升了约 1.4 个百分点的 F1，验证了旋转位置编码在保留相对位置信息上的优势，但仍落后于无位置编码的方案。这表明更先进的位置编码有助于缓解绝对位置编码的副作用，但并未完全消除。

因果 mask 的不适用性 因果关系 mask 强制模型单向关注，直接导致模型无法利用后文信息，F1 骤降至 0.4871，证明情感分析任务需要全局上下文，双向注意力是合理选择。

超参数敏感性 超参数搜索揭示了 Transformer 在该任务上的脆弱性：

- **模型容量**：过大的嵌入维度（512）或过深的层数（6）会导致训练崩溃，而适中配置（128/256 维，2 4 层）性能优异且稳健。这表明在小规模数据上，轻量化设计是防止过拟合和优化困难的关键。

- **注意力头数**: 4 16 头之间性能稳定, 但头数过少 (2) 或某些特定配置 (8 头) 会出现波动, 应综合考虑每头维度与模型整体容量的匹配。
- **学习率**: 学习率 1×10^{-4} 更稳定, 稍大的 5×10^{-4} 也可接受, 但 1×10^{-3} 直接导致发散。建议在训练 Transformer 时采用较低的学习率或配合学习率预热策略。
- **Dropout**: 标准 Dropout (0.1) 或完全不用 Dropout 均可, 但较高的 Dropout 率 (0.2) 会严重阻碍学习, 尤其当训练轮次较少时。因此应避免过强的正则化, 或相应增加训练轮次以补偿。

6 总结

本实验系统地比较了 Attention 与 RNN 在 IMDB 情感分类上的表现, 并探究了多种位置编码策略、mask 模式以及关键超参数的影响。主要结论如下:

1. LSTM 在该任务上略优于标准 Transformer, 训练更稳定;
2. 位置编码并非普遍有益, 在本实验设置下甚至可能降低性能, 无位置编码模型表现最佳;
3. 手工实现 MHA 与官方库性能持平;
4. RoPE 优于正弦编码但仍不及无位置编码;
5. 因果 mask 严重损害分类性能, 双向注意力不可或缺;
6. 模型容量与学习率对 Transformer 性能影响显著, 小而精的配置 (浅层、低维、小学习率) 更适合小规模情感分类任务。

这些发现为文本分类任务中的模型设计提供了有价值的参考: 对于简单分类, 复杂的序列建模组件需经过验证, 适度简化模型并谨慎调整超参数往往能获得更好且更稳定的性能。

7 代码文件结构说明

为方便审阅, 代码目录结构如下:

```

1  .
2  main.py           # 实验入口, 命令行参数解析
3  data/
4      imdb_sentiment_data.csv
5  results/         # 输出结果、模型、图片

```



```

6      src/
7          config.py          # 实验配置类
8          dataset.py        # 数据加载、IMDBDataset
9          experiment_runner.py # 核心运行逻辑
10         tokenizer.py       # 词表构建与编码
11         trainer.py         # 训练/评估循环
12         utils.py           # 位置编码、随机种子等
13         models/
14             attention_model.py
15             rnn_model.py
16             manual_mha.py
17             attention_manual_model.py

```

参考文献

1. Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]. NeurIPS, 2017.
2. Su J, Lu Y, Pan S, et al. RoFormer: Enhanced Transformer with Rotary Position Embedding[J]. arXiv:2104.09864, 2021.

原始输出

超参数实验:

命令:

```

!python /kaggle/input/main.py --exp hyper_head
!python /kaggle/input/main.py --exp hyper_embed
!python /kaggle/input/main.py --exp hyper_layers
!python /kaggle/input/main.py --exp hyper_lr
!python /kaggle/input/main.py --exp hyper_dropout

```

输出:

Dataset loaded. Total samples: 50000

Dataset Statistics:

Total samples: 50000

Positive samples: 25000

Negative samples: 25000

Text length statistics:

Mean: 231.16

Median: 173.00

Max: 2470

95th percentile: 590.00

Data split:

Train: 36000, Val: 4000, Test: 10000

Beginning training with config: ExperimentConfig(exp_type=<ExperimentType.HYPERPARAM_

--- Training with num_heads=2 ---

Epoch 1: val_acc=0.7692, val_f1=0.7577

Epoch 2: val_acc=0.7622, val_f1=0.7859

Epoch 3: val_acc=0.7425, val_f1=0.7744

Epoch 4: val_acc=0.7548, val_f1=0.7650

Epoch 5: val_acc=0.7127, val_f1=0.7554

num_heads=2: Best Val F1=0.7859

--- Training with num_heads=4 ---

Epoch 1: val_acc=0.7480, val_f1=0.7262

Epoch 2: val_acc=0.8065, val_f1=0.8181

Epoch 3: val_acc=0.7880, val_f1=0.7542

Epoch 4: val_acc=0.8227, val_f1=0.8311

Epoch 5: val_acc=0.8317, val_f1=0.8327

num_heads=4: Best Val F1=0.8327

--- Training with num_heads=8 ---

Epoch 1: val_acc=0.6777, val_f1=0.7109

Epoch 2: val_acc=0.6615, val_f1=0.6492

Epoch 3: val_acc=0.6055, val_f1=0.4719

Epoch 4: val_acc=0.4998, val_f1=0.6664

Epoch 5: val_acc=0.4998, val_f1=0.6664

num_heads=8: Best Val F1=0.7109

--- Training with num_heads=16 ---

Epoch 1: val_acc=0.7738, val_f1=0.8024

Epoch 2: val_acc=0.8273, val_f1=0.8360
Epoch 3: val_acc=0.8395, val_f1=0.8425
Epoch 4: val_acc=0.8223, val_f1=0.8066
Epoch 5: val_acc=0.8355, val_f1=0.8388
num_heads=16: Best Val F1=0.8425

Summary of hyperparameter experiment:

num_heads=2, F1=0.7859
num_heads=4, F1=0.8327
num_heads=8, F1=0.7109
num_heads=16, F1=0.8425

Dataset loaded. Total samples: 50000

Dataset Statistics:

Total samples: 50000
Positive samples: 25000
Negative samples: 25000

Text length statistics:

Mean: 231.16
Median: 173.00
Max: 2470
95th percentile: 590.00

Data split:

Train: 36000, Val: 4000, Test: 10000

Beginning training with config: ExperimentConfig(exp_type=<ExperimentType.HYPERPARAM_

--- Training with embed_dim=128 ---

Epoch 1: val_acc=0.8043, val_f1=0.7895
Epoch 2: val_acc=0.8313, val_f1=0.8378
Epoch 3: val_acc=0.8385, val_f1=0.8442
Epoch 4: val_acc=0.8460, val_f1=0.8517
Epoch 5: val_acc=0.8427, val_f1=0.8477
embed_dim=128: Best Val F1=0.8517

--- Training with embed_dim=256 ---

Epoch 1: val_acc=0.7977, val_f1=0.8168
Epoch 2: val_acc=0.8317, val_f1=0.8376
Epoch 3: val_acc=0.8442, val_f1=0.8441
Epoch 4: val_acc=0.8475, val_f1=0.8488
Epoch 5: val_acc=0.8430, val_f1=0.8476
embed_dim=256: Best Val F1=0.8488

--- Training with embed_dim=512 ---

Epoch 1: val_acc=0.5002, val_f1=0.0000
Epoch 2: val_acc=0.4998, val_f1=0.6664
Epoch 3: val_acc=0.4998, val_f1=0.6664
Epoch 4: val_acc=0.5002, val_f1=0.0000
Epoch 5: val_acc=0.4998, val_f1=0.6664
embed_dim=512: Best Val F1=0.6664

Summary of hyperparameter experiment:

embed_dim=128, F1=0.8517
embed_dim=256, F1=0.8488
embed_dim=512, F1=0.6664

Dataset loaded. Total samples: 50000

Dataset Statistics:

Total samples: 50000
Positive samples: 25000
Negative samples: 25000

Text length statistics:

Mean: 231.16
Median: 173.00
Max: 2470
95th percentile: 590.00

Data split:

Train: 36000, Val: 4000, Test: 10000

Beginning training with config: ExperimentConfig(exp_type=<ExperimentType.HYPERPARAM_

--- Training with num_layers=2 ---

Epoch 1: val_acc=0.8073, val_f1=0.7895
Epoch 2: val_acc=0.8235, val_f1=0.8405
Epoch 3: val_acc=0.8413, val_f1=0.8524
Epoch 4: val_acc=0.8525, val_f1=0.8499
Epoch 5: val_acc=0.8435, val_f1=0.8377
num_layers=2: Best Val F1=0.8524

--- Training with num_layers=4 ---

Epoch 1: val_acc=0.7615, val_f1=0.7244
Epoch 2: val_acc=0.8257, val_f1=0.8372
Epoch 3: val_acc=0.8345, val_f1=0.8239
Epoch 4: val_acc=0.8240, val_f1=0.8417
Epoch 5: val_acc=0.8413, val_f1=0.8400
num_layers=4: Best Val F1=0.8417

--- Training with num_layers=6 ---

Epoch 1: val_acc=0.4998, val_f1=0.6664
Epoch 2: val_acc=0.5002, val_f1=0.0000
Epoch 3: val_acc=0.5002, val_f1=0.0000
Epoch 4: val_acc=0.5002, val_f1=0.0000
Epoch 5: val_acc=0.4998, val_f1=0.6664
num_layers=6: Best Val F1=0.6664

Summary of hyperparameter experiment:

num_layers=2, F1=0.8524

num_layers=4, F1=0.8417

num_layers=6, F1=0.6664

Dataset loaded. Total samples: 50000

Dataset Statistics:

Total samples: 50000

Positive samples: 25000

Negative samples: 25000

Text length statistics:

Mean: 231.16

Median: 173.00

Max: 2470

95th percentile: 590.00

Data split:

Train: 36000, Val: 4000, Test: 10000

Beginning training with config: ExperimentConfig(exp_type=<ExperimentType.HYPERPARAM_

--- Training with lr=0.001 ---

Epoch 1: val_acc=0.4998, val_f1=0.6664

Epoch 2: val_acc=0.4998, val_f1=0.6664

Epoch 3: val_acc=0.4998, val_f1=0.6664

Epoch 4: val_acc=0.5002, val_f1=0.0000

Epoch 5: val_acc=0.4998, val_f1=0.6664

lr=0.001: Best Val F1=0.6664

--- Training with lr=0.0005 ---

Epoch 1: val_acc=0.7920, val_f1=0.7779

Epoch 2: val_acc=0.8185, val_f1=0.8300

Epoch 3: val_acc=0.8050, val_f1=0.7830

Epoch 4: val_acc=0.8245, val_f1=0.8337

Epoch 5: val_acc=0.8400, val_f1=0.8434

lr=0.0005: Best Val F1=0.8434

--- Training with lr=0.0001 ---

Epoch 1: val_acc=0.7530, val_f1=0.7852

Epoch 2: val_acc=0.7823, val_f1=0.8088

Epoch 3: val_acc=0.8303, val_f1=0.8290

Epoch 4: val_acc=0.8337, val_f1=0.8302

Epoch 5: val_acc=0.8385, val_f1=0.8457

lr=0.0001: Best Val F1=0.8457

Summary of hyperparameter experiment:

lr=0.001, F1=0.6664

lr=0.0005, F1=0.8434

lr=0.0001, F1=0.8457

Dataset loaded. Total samples: 50000

Dataset Statistics:

Total samples: 50000

Positive samples: 25000

Negative samples: 25000

Text length statistics:

Mean: 231.16

Median: 173.00

Max: 2470

95th percentile: 590.00

Data split:

Train: 36000, Val: 4000, Test: 10000

Beginning training with config: ExperimentConfig(exp_type=<ExperimentType.HYPERPARAM_

--- Training with dropout=0.0 ---

Epoch 1: val_acc=0.7812, val_f1=0.7576

Epoch 2: val_acc=0.8170, val_f1=0.8351

Epoch 3: val_acc=0.8393, val_f1=0.8455

Epoch 4: val_acc=0.8233, val_f1=0.8018

Epoch 5: val_acc=0.8433, val_f1=0.8389

dropout=0.0: Best Val F1=0.8455

--- Training with dropout=0.1 ---

Epoch 1: val_acc=0.7542, val_f1=0.7077

Epoch 2: val_acc=0.7960, val_f1=0.8199

Epoch 3: val_acc=0.8057, val_f1=0.7789

Epoch 4: val_acc=0.8427, val_f1=0.8486

Epoch 5: val_acc=0.8250, val_f1=0.8095

dropout=0.1: Best Val F1=0.8486

--- Training with dropout=0.2 ---

Epoch 1: val_acc=0.5875, val_f1=0.7016

Epoch 2: val_acc=0.6910, val_f1=0.6811

Epoch 3: val_acc=0.6705, val_f1=0.6856

Epoch 4: val_acc=0.6388, val_f1=0.7073

Epoch 5: val_acc=0.6643, val_f1=0.6500

dropout=0.2: Best Val F1=0.7073

Summary of hyperparameter experiment:

dropout=0.0, F1=0.8455

dropout=0.1, F1=0.8486

dropout=0.2, F1=0.7073

其他实验:

命令行:

```
!python /kaggle/input/main.py --exp attention
```

```
!python /kaggle/input/main.py --exp rnn
```

```
!python /kaggle/input/main.py --exp manual_mha
```

```
!python /kaggle/input/main.py --exp pos_encoding
```

```
!python /kaggle/input/main.py --exp rope
```

```
!python /kaggle/input/main.py --exp causal_mask
```

输出:

Dataset loaded. Total samples: 50000

Dataset Statistics:

Total samples: 50000

Positive samples: 25000

Negative samples: 25000

Text length statistics:

Mean: 231.16

Median: 173.00

Max: 2470

95th percentile: 590.00

Data split:

Train: 36000, Val: 4000, Test: 10000

Beginning training with config: ExperimentConfig(exp_type=<ExperimentType.TRAIN: 1>,

Epoch 1: train_loss=0.5828, val_acc=0.7588, val_f1=0.7228

Epoch 2: train_loss=0.4061, val_acc=0.8137, val_f1=0.8325

Epoch 3: train_loss=0.3296, val_acc=0.8400, val_f1=0.8485

Epoch 4: train_loss=0.2801, val_acc=0.8413, val_f1=0.8370

Epoch 5: train_loss=0.2408, val_acc=0.8430, val_f1=0.8377

Test Result: loss=0.3960, acc=0.8385, f1=0.8365

Dataset loaded. Total samples: 50000

Dataset Statistics:

Total samples: 50000

Positive samples: 25000

Negative samples: 25000

Text length statistics:

Mean: 231.16

Median: 173.00

Max: 2470

95th percentile: 590.00

Data split:

Train: 36000, Val: 4000, Test: 10000

Beginning training with config: ExperimentConfig(exp_type=<ExperimentType.TRAIN: 1>,

Epoch 1: train_loss=0.4740, val_acc=0.8387, val_f1=0.8381

Epoch 2: train_loss=0.3155, val_acc=0.8602, val_f1=0.8649

Epoch 3: train_loss=0.2388, val_acc=0.8620, val_f1=0.8628

Epoch 4: train_loss=0.1718, val_acc=0.8662, val_f1=0.8666

Epoch 5: train_loss=0.1069, val_acc=0.8612, val_f1=0.8633

Test Result: loss=0.4938, acc=0.8589, f1=0.8622

Dataset loaded. Total samples: 50000

Dataset Statistics:

Total samples: 50000

Positive samples: 25000

Negative samples: 25000

Text length statistics:

Mean: 231.16

Median: 173.00

Max: 2470

95th percentile: 590.00

Data split:

Train: 36000, Val: 4000, Test: 10000

Beginning training with config: ExperimentConfig(exp_type=<ExperimentType.COMPARISON:

--- Training Sinusoidal PE ---

Epoch 1: train_loss=0.5437, val_acc=0.8077, val_f1=0.7960

Epoch 2: train_loss=0.3860, val_acc=0.8305, val_f1=0.8188

Epoch 3: train_loss=0.3186, val_acc=0.8350, val_f1=0.8404

Epoch 4: train_loss=0.2693, val_acc=0.8310, val_f1=0.8261

Epoch 5: train_loss=0.2319, val_acc=0.8327, val_f1=0.8391

Sinusoidal PE Best Val F1: 0.8404

--- Training Manual MHA ---

Epoch 1: train_loss=0.5939, val_acc=0.7655, val_f1=0.7942

Epoch 2: train_loss=0.3974, val_acc=0.7837, val_f1=0.7455

Epoch 3: train_loss=0.3375, val_acc=0.8170, val_f1=0.8330

Epoch 4: train_loss=0.2955, val_acc=0.8397, val_f1=0.8443

Epoch 5: train_loss=0.2763, val_acc=0.8377, val_f1=0.8443

Manual MHA Best Val F1: 0.8443

Dataset loaded. Total samples: 50000

Dataset Statistics:

Total samples: 50000

Positive samples: 25000

Negative samples: 25000

Text length statistics:

Mean: 231.16

Median: 173.00

Max: 2470

95th percentile: 590.00

Data split:

Train: 36000, Val: 4000, Test: 10000

Beginning training with config: ExperimentConfig(exp_type=<ExperimentType.COMPARISON:

--- Training Sinusoidal PE ---

Epoch 1: train_loss=0.6618, val_acc=0.7005, val_f1=0.7024

Epoch 2: train_loss=0.5202, val_acc=0.7630, val_f1=0.7469
Epoch 3: train_loss=0.4488, val_acc=0.7853, val_f1=0.7865
Epoch 4: train_loss=0.4159, val_acc=0.7855, val_f1=0.7840
Epoch 5: train_loss=0.4428, val_acc=0.7670, val_f1=0.7666
Sinusoidal PE Best Val F1: 0.7865

--- Training No Pos ---

Epoch 1: train_loss=0.4953, val_acc=0.8235, val_f1=0.8146
Epoch 2: train_loss=0.3505, val_acc=0.8430, val_f1=0.8476
Epoch 3: train_loss=0.2984, val_acc=0.8413, val_f1=0.8309
Epoch 4: train_loss=0.2636, val_acc=0.8550, val_f1=0.8564
Epoch 5: train_loss=0.2398, val_acc=0.8555, val_f1=0.8571
No Pos Best Val F1: 0.8571

Dataset loaded. Total samples: 50000

Dataset Statistics:

Total samples: 50000

Positive samples: 25000

Negative samples: 25000

Text length statistics:

Mean: 231.16

Median: 173.00

Max: 2470

95th percentile: 590.00

Data split:

Train: 36000, Val: 4000, Test: 10000

Beginning training with config: ExperimentConfig(exp_type=<ExperimentType.COMPARISON:

--- Training Sinusoidal PE ---

Epoch 1: train_loss=0.6556, val_acc=0.7192, val_f1=0.7283
Epoch 2: train_loss=0.5010, val_acc=0.8025, val_f1=0.8087
Epoch 3: train_loss=0.4099, val_acc=0.8157, val_f1=0.8113
Epoch 4: train_loss=0.3537, val_acc=0.8073, val_f1=0.8197
Epoch 5: train_loss=0.3725, val_acc=0.7945, val_f1=0.7792
Sinusoidal PE Best Val F1: 0.8197

--- Training RoPE ---

Epoch 1: train_loss=0.5537, val_acc=0.7935, val_f1=0.7973

Epoch 2: train_loss=0.3983, val_acc=0.8140, val_f1=0.8178

Epoch 3: train_loss=0.3521, val_acc=0.8247, val_f1=0.8219

Epoch 4: train_loss=0.3246, val_acc=0.8293, val_f1=0.8301

Epoch 5: train_loss=0.3073, val_acc=0.8120, val_f1=0.7957

RoPE Best Val F1: 0.8301

Dataset loaded. Total samples: 50000

Dataset Statistics:

Total samples: 50000

Positive samples: 25000

Negative samples: 25000

Text length statistics:

Mean: 231.16

Median: 173.00

Max: 2470

95th percentile: 590.00

Data split:

Train: 36000, Val: 4000, Test: 10000

Beginning training with config: ExperimentConfig(exp_type=<ExperimentType.COMPARISON:

--- Training Sinusoidal PE ---

Epoch 1: train_loss=0.6549, val_acc=0.7170, val_f1=0.7268

Epoch 2: train_loss=0.5094, val_acc=0.7770, val_f1=0.7925

Epoch 3: train_loss=0.4500, val_acc=0.7953, val_f1=0.7937

Epoch 4: train_loss=0.4428, val_acc=0.7808, val_f1=0.7970

Epoch 5: train_loss=0.4308, val_acc=0.7880, val_f1=0.7722

Sinusoidal PE Best Val F1: 0.7970

--- Training Causal mask ---

Epoch 1: train_loss=0.5384, val_acc=0.6122, val_f1=0.4403

Epoch 2: train_loss=0.3918, val_acc=0.6210, val_f1=0.4871

Epoch 3: train_loss=0.3203, val_acc=0.6220, val_f1=0.4553

Epoch 4: train_loss=0.2701, val_acc=0.6112, val_f1=0.4860
Epoch 5: train_loss=0.2333, val_acc=0.6210, val_f1=0.4523
Causal mask Best Val F1: 0.4871